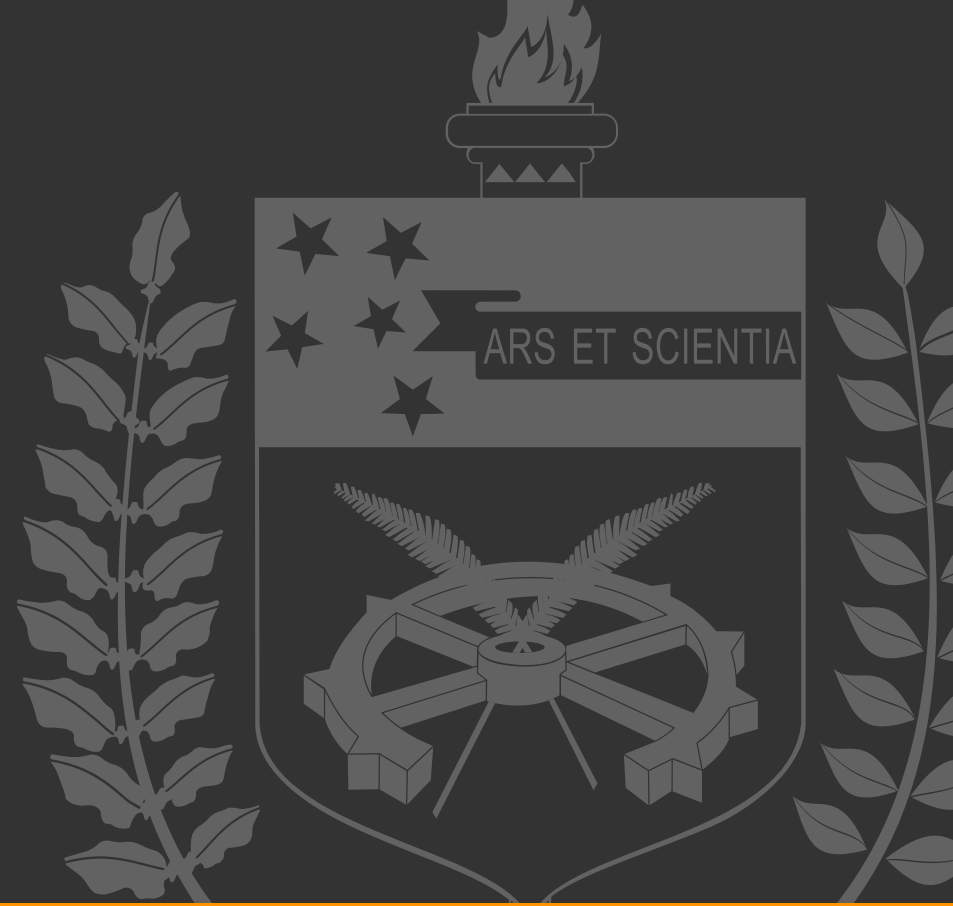


Programação Orientada a Objetos

Classes, objetos, atributos e métodos



Universidade Federal de Santa Catarina – UFSC

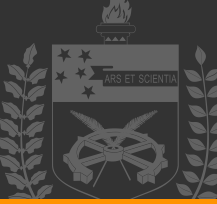
Centro Tecnológico de Joinville – CTJ

Prof. Arthur Gabardo

2026.1

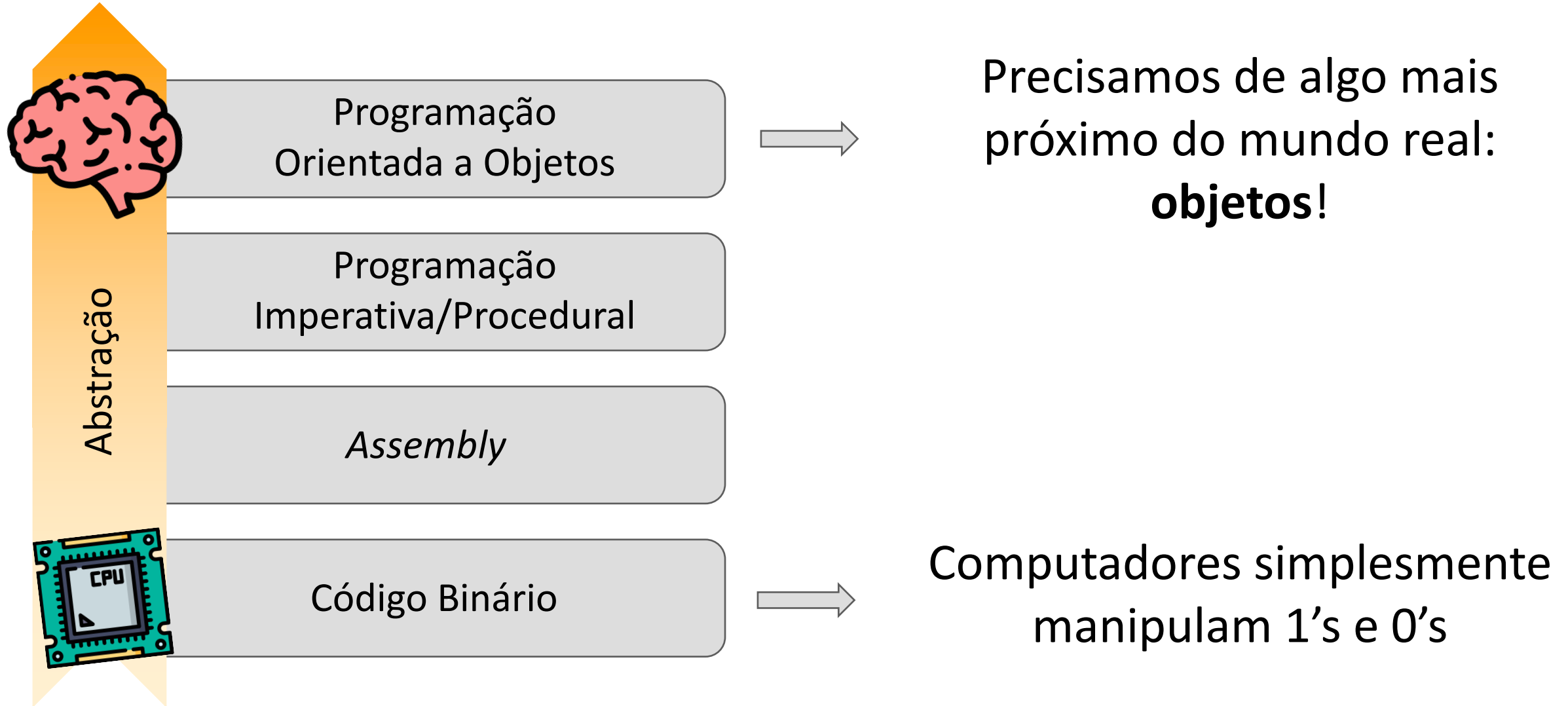


- Na aula passada discutimos alguns aspectos gerais da linguagem C++
 - Extensão dos tipos primitivos de C (bool, auto, etc.)
 - Diferenças na alocação dinâmica (new e delete)
 - Versões:
 - C++11, C++17, C++20, C++23, ...
 - Compiladores:
 - GCC (g++), Clang, MSVC
- Principal característica do C++ → Orientação a Objetos

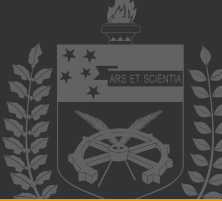


- Introdução
- Princípios de OOP
- Classes e Objetos
- Implementação em C++

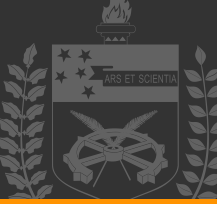
Por que Orientação a Objetos?



Programação Orientada a Objetos



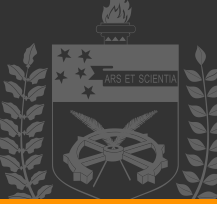
- **Programação Orientada a Objetos** (OOP – sigla em inglês) é um paradigma de análise, projeto e programação de sistemas de software
 - OOP foca na modularidade e escalabilidade de código
- Modela sistemas complexos e dinâmicos por meio da abstração das características (atributos) e comportamentos (métodos) dos elementos envolvidos
- Cada um dos elementos constitui um objeto
 - Diferentemente da programação estruturada, os objetos integram as características e comportamentos em uma coisa só



- Em uma loja online, um cliente pode **adicionar** produtos ao seu carrinho. Cada produto tem nome, preço e quantidade em estoque. O carrinho deve **calcular o valor** total da compra e permitir a **remoção de itens**.
- Quais os objetos?
- Quais as características?
- Quais são os comportamentos?



- Um professor atribui notas a alunos em uma disciplina no Moodle. Cada aluno tem um nome e uma matrícula. O sistema calcula a média das notas dos alunos e determina se estão aprovados.
- Por que não escolhemos outras representações?
 - É dever do programador escolher uma boa representação dos elementos que constituem o problema



A Orientação a Objetos se baseia em quatro princípios fundamentais

- **Abstração**
- **Encapsulamento**
- Herança
- Polimorfismo
 - Mais sobre na próxima aula



- Técnica de simplificação:
 - Representar algo complexo mostrando só o que importa
 - Definir uma **interface** para utilização de um biblioteca ou programa
- Base para projetar sistemas organizados, modulares e reutilizáveis
- Em POO, significa criar modelos que focam no essencial de um objeto ou conceito
- Abstrair é se importar com ***o que*** ao invés de ***como***

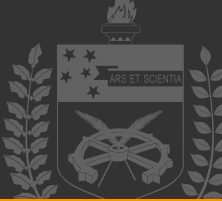


Pense num aplicativo de streaming de música

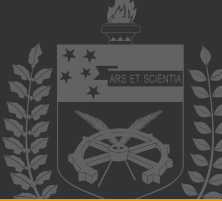
- Você pode escolher a música que quer escutar, dar play/pause, criar e compartilhar playlists
 - Não é preciso entender como funciona o algoritmo de compressão ou o protocolo de rede
 - Imagine se fosse necessário informar:
 - Bitrate desejada para o streaming
 - Codec de compressão
 - IP do servidor que contém a música
- Um bom sistema abstrai os detalhes técnicos por meio de sua interface
 - Quando há falta de abstração o usuário lida com o que não devia



- Uma boa abstração também é essencial para permitir a escalabilidade e reutilização de código
 - Facilita adicionar novas funcionalidades sem reescrever tudo
 - Evita “reinventar a roda”
 - Ligado aos conceitos de herança e polimorfismo
- Uma boa interface torna a implementação mais robusta
 - Mudanças internas ficam isoladas → não quebram o restante do sistema
- Deixa o código mais legível, organizado e modular



- Objetos “encapsulam” tanto comportamentos como características
 - Diferente da programação estruturada onde funções agiam sobre os dados
 - Comportamentos e características separadas
- Encapsular permite esconder determinadas características de um objeto
- Restringir acesso por meio de métodos controlados
 - Evita que atributos sejam modificados de forma incorreta
 - Preserva o estado e consistência dos objetos



Funciona como um cofre ou caixa eletrônico

- Em um caixa eletrônico você pode sacar, depositar, consultar saldo
 - Entretanto, estas ações são feitas por meio de operações seguras
 - Não é possível simplesmente abrir o cofre e pegar dinheiro direto
- Um mau encapsulamento é como deixar o cofre aberto
 - Mudanças no estado dos objetos podem ser feitas de maneiras arbitrárias
 - Não há controle ou segurança nas operações realizadas



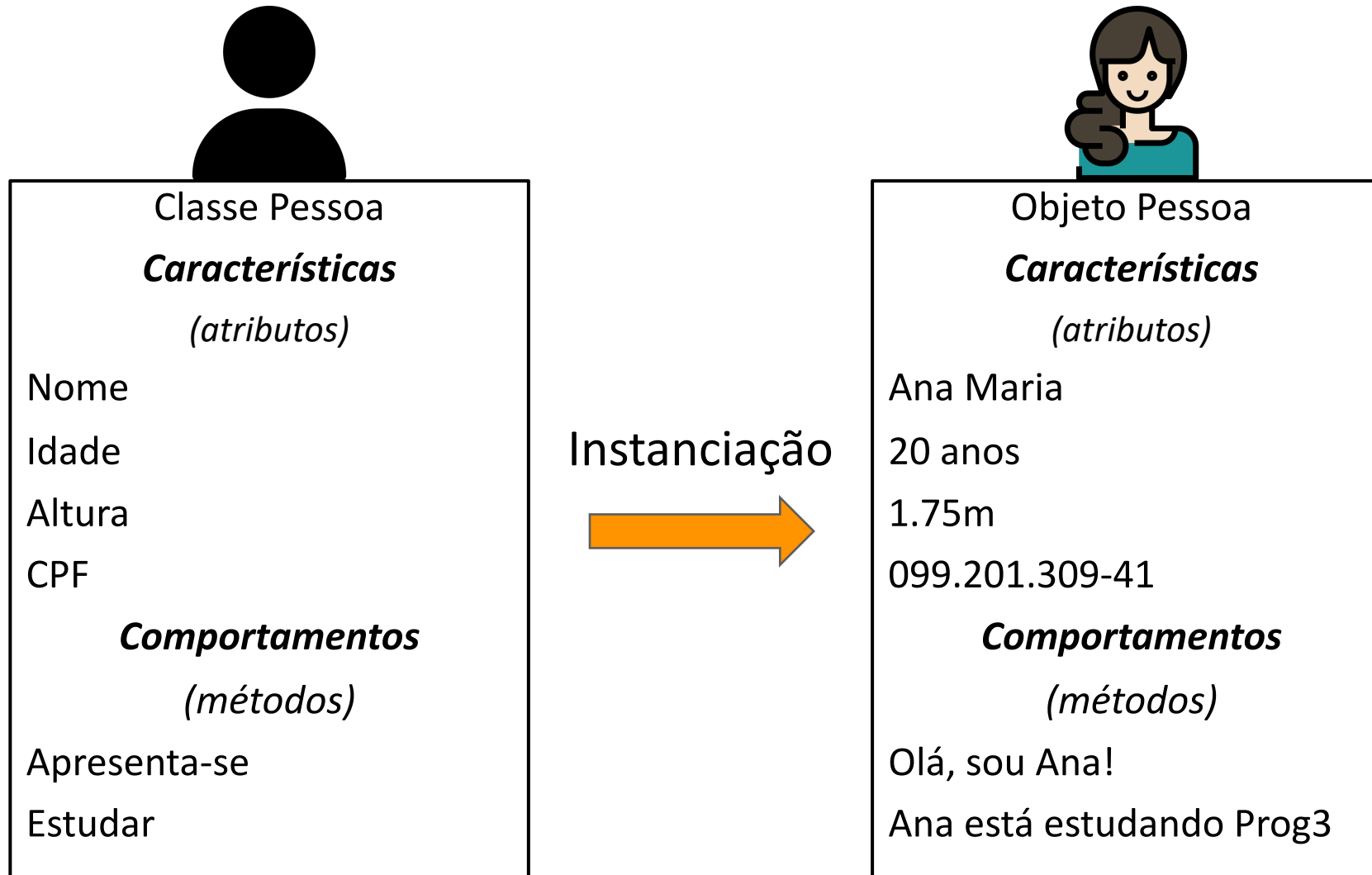
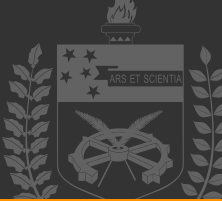
- Até o momento temos falado bastante sobre objetos, mas o que são de fato?
 - Objetos são instâncias de **classes**
 - Precisamos dar um passo para trás
- Uma classe é um molde, uma receita, um projeto
 - Definem as abstrações e como é feito o encapsulamento
 - Possuem métodos (funções membro) e atributos (dados membro)
- As classes são os elementos centrais da POO

Classes e Objetos



- A classe é uma ideia e o objeto a realidade
 - Classe = planta baixa → Objeto = casa construída
- Objetos possuem valores próprios para seus atributos
 - O conjunto de atributos inicializados é denominado de **estado** do objeto
- São os objetos que de fato executam os comportamentos definidos pela classe por meio das funções membro (métodos)
- Uma mesma classe pode gerar diversos objetos distintos

Classes e Objetos





- Os atributos são os dados abstraídos pelas classes e armazenados pelos objetos
 - O conjunto de atributos inicializado forma o estado do objeto
- De maneira geral podem ser:
 - **Públicos** (acesso direto)
 - **Privados** (acesso restrito)
- É boa prática de encapsulamento que os atributos sejam privados sempre que possível
- O acesso restrito é feito por meio de métodos públicos



- Assim como os atributos, podem ser públicos ou privados
- São funções definidas no escopo da classe e definem os comportamentos dos objetos
- Podem acessar e modificar atributos privados da mesma classe , bem como invocar funções membro privadas
- Ajudam a manter o estado dos objetos consistentes e válidos
 - Princípio do encapsulamento

Exemplo: classe Aluno em C++



Veja a *declaração* da classe pessoa:

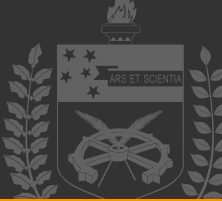
```
#include <string>

class Pessoa {
private:
    std::string nome;
    unsigned int idade;
    float altura;
    int cpf[11];

public:
    void apresentar() const;
    void estudar() const;
};
```

- A definição de classes em C++ é feita pela palavra-chave `class` seguida do nome da classe
- Especificamos a restrição de acesso usando as palavras chaves `private`, `public` e `protected`
 - Caso especificador não seja explicitado → `private`
- Atributos são declarados como variáveis
- Métodos são declarados como funções

Declaração vs. Definição



Declaração (arquivo .h)

- Lista de atributos e métodos da classe
- Mostra o que a classe promete oferecer
- Funciona como um contrato
 - Define a interface da classe



Definição (arquivo .cpp)

- Contém a implementação dos métodos declarados
- Especifica o comportamento real da classe

Declaração vs. Definição



pessoa.h

```
#include <string>

class Pessoa {
private:
    std::string nome;
    unsigned int idade;
    float altura;
    int cpf[11];

public:
    void apresentar() const;
    void estudar() const;
};
```

pessoa.cpp

```
#include <iostream>
#include "pessoa.h"

void Pessoa::apresentar() const {
    std::cout << "Olá, me chamo " << nome
               << ", tenho " << idade
               << " anos e " << altura
               << "m de altura.\n";
}

void Pessoa::estudar() const {
    std::cout << nome << " esta estudando.\n";
}
```

- A inclusão de arquivos .h é análoga à linguagem C
- Necessário especificar o escopo na definição das funções membro (::)
 - `retorno nome_classe::nome_metodo() {...}`

Modificando atributos privados



- Como vimos, a modificação dos atributos privados só pode ser feita por funções membro da mesma classe
- É comum o uso de **getters** e **setters**
 - Funções de acesso e modificação
 - Garantem a consistência dos objetos

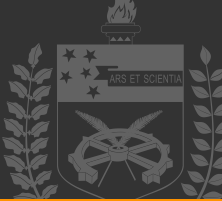
```
Pessoa &setNome(std::string _nome);  
std::string getNome() const;
```

```
Pessoa& Pessoa::setNome(std::string _nome) {  
    // verifica se o nome é valido ...  
    this->nome = _nome;  
    return *this; //this é um ponteiro para si mesmo  
}
```

```
std::string Pessoa::getNome() const { return nome; }
```

← Retorna uma referência
ao próprio objeto, permite
chamadas aninhadas

← Função membro
constante não altera o
estado do objeto



- Como instanciar classes para criar objetos?

```
int main() {  
    Pessoa a;  
    a.setNome("Joao").setAltura(1.80).setIdade(20);  
    a.apresentar();  
    return 0;  
}
```

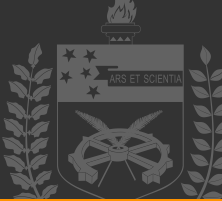
Olá, me chamo Joao, tenho 20 anos e 1.8m de altura.

- Definir cada um dos atributos individualmente é trabalhoso e verboso
- Podemos usar **construtores** para instanciar uma classe e inicializar seus atributos simultaneamente
 - Essencial se houverem atributos constantes



- Os construtores são funções membros especiais da classe
 - Sempre têm o mesmo nome que a classe
 - Não possuem nenhum retorno, nem mesmo void
 - São sempre chamados automaticamente ao se criar um objeto
- Vamos ver três tipos de construtores
 - Padrão `Pessoa();`
 - Parametrizado `Pessoa(std::string _nome, float altura, unsigned int idade);`
 - Cópia `Pessoa(const Pessoa& p);`
- Caso nenhum construtor seja declarado/definido, o compilador gera um construtor padrão automaticamente

Construtor padrão



- O construtor padrão é usado para inicializar os campos (atributos) de um objeto com valores padrão

```
class Pessoa {  
    private:  
        const std::string nome;  
        float altura;  
        unsigned int idade;  
  
    public:  
        Pessoa() {  
            this->nome = "Alice";  
            this->altura = 1.75;  
            this->idade = 20;  
        }  
};  
  
int main() {  
    Pessoa alice;  
    // nome = Alice, altura = 1.75, idade = 20  
}
```

Lista de inicialização



- E se a classe possuir atributos constantes?

```
class Pessoa {  
    private:  
        const std::string nome;  
        float altura;  
        unsigned int idade;  
  
    public:  
        Pessoa() {  
            this->nome = "Alice"; // Erro! Modificando const  
            this->altura = 1.75;  
            this->idade = 20;  
        }  
};
```

- Neste caso usamos uma **lista de inicialização** no construtor
 - Não limitado ao construtor padrão

Lista de inicialização



- Com a lista de inicialização podemos inicializar atributos de uma classe diretamente no cabeçalho do construtor, antes do corpo { }
- Usar a lista de inicialização:
 - Evita atribuições extras
 - Permite inicializar atributos const ou referências

```
class Pessoa {  
    private:  
        const std::string nome;  
        float altura;  
        unsigned int idade;  
  
    public:  
        Pessoa() : nome("Bob"), altura(1.8), idade(30) {}  
};
```

Construtor parametrizado



- Permite criar o objeto com valores especificados

```
class Pessoa {  
    private:  
        const std::string nome;  
        float altura;  
        unsigned int idade;  
  
    public:  
        Pessoa(std::string _nome, float _altura, unsigned int _idade) : nome(_nome) {  
            this->altura = _altura;  
            this->idade = _idade;  
        }  
};  
  
int main() {  
    Pessoa b = Pessoa("Bob", 1.8, 30);  
    // nome = Bob, altura = 1.8, idade = 30  
}
```

Construtor de cópia



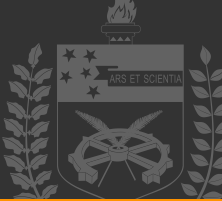
- O construtor de cópia permite criar objetos idênticos a partir de um já instanciado

```
class Pessoa {  
    private:  
        const std::string nome;  
        float altura;  
        unsigned int idade;  
  
    public:  
        Pessoa() : nome("Bob"), altura(1.8), idade(30) {};  
        Pessoa(const Pessoa &p) : nome(p.nome), altura(p.altura), idade(p.idade) {}  
};  
  
int main() {  
    Pessoa a;  
    Pessoa b = Pessoa(a);  
    return 0;  
}
```



- Assim como os construtores, destrutores são métodos especiais que são chamados automaticamente no fim de vida de um objeto
- É usado para liberar recursos (memória, arquivos, conexões, etc.)
- Assim como os construtores, destrutores:
 - Tem o mesmo nome da classe, mas precedido de ~
 - `~Pessoa()`;
- Destrutores não recebem parâmetros nem possuem retornos

Destrutores



Qual a saída deste programa?

```
class Pessoa {
private:
    const std::string nome;
    float altura;
    unsigned int idade;

public:
    Pessoa(std::string _nome, float _altura, unsigned int _idade) :
        nome(_nome) {
        this->altura = _altura;
        this->idade = _idade;
        std::cout << "Objeto criado : " << nome << std::endl;
    }
    ~Pessoa() { std::cout << "Objeto destruido : " << nome << std::endl;}
};

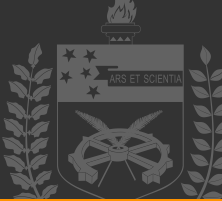
int main() {
    Pessoa a = Pessoa("Bob", 1.8, 30);
    Pessoa *b = new Pessoa("Alice", 1.75, 20);
    delete b;
    return 0;
}
```

Métodos e atributos estáticos



- São membros compartilhados entre todos os objetos da mesma classe
 - Pertencem à classe, e não a cada um dos objetos
- Atributos estáticos:
 - São compartilhados entre todas as instâncias
 - Devem ser inicializados fora da classe/construtor
- Métodos estáticos:
 - Podem ser chamados sem criar nenhum objeto da classe
 - Acessa apenas os atributos estáticos de uma classe
 - Podem ser úteis para obter informações sobre todas as instâncias de uma classe
 - Contar instâncias inicializadas, soma de todos os atributos, ...

Métodos e atributos estáticos



```
class Pessoa {
private:
    const std::string nome;
    float altura;
    unsigned int idade;
    static unsigned int contPessoas;           // atributo estático

public:
    Pessoa(std::string _nome, float _altura=1.7, unsigned int _idade=20) :
        nome(_nome), altura(_altura), idade(_idade) {contPessoas++;}
    ~Pessoa() {contPessoas--;}
    static unsigned int getTotalPessoas();    // método estático
};

unsigned int Pessoa::contPessoas = 0;
unsigned int Pessoa::getTotalPessoas() {return contPessoas;}

int main() {
    Pessoa a = Pessoa("Alice");
    Pessoa *b = new Pessoa("Bob");
    std::cout << "Total de pessoas: " << Pessoa::getTotalPessoas() << std::endl;
    delete b;
    std::cout << "Total de pessoas: " << Pessoa::getTotalPessoas() << std::endl;
    return 0;
}
```

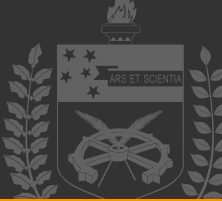


Vamos implementar a classe Pessoa



Implemente uma classe ContaBancaria em C++ que:

1. Possua atributos privados: número da conta, nome do titular, saldo
2. Implemente construtores padrão, parametrizado e de cópia
3. Utilize lista de inicialização nos construtores
4. Implemente um destrutor que exiba titular e num. ao encerrar a conta
5. Crie métodos públicos: depositar, sacar, exhibirSaldo, trasferirTitularidade
6. Utilize um atributo estático para contabilizar o saldo de todas as contas
7. Implemente um método estático que retorne este saldo



- Leitura recomendada: Capítulo 3 do *Deitel – C++ Como Programar*
- Trabalhos: Definição dos grupos disponível no Moodle
- Seminários: Escolher uma linguagem orientada a objetos
 - Apresentar: história, funcionalidades/sintaxe, exemplos e aplicações
 - Detalhes completos no Moodle
- Próxima aula: Herança, Composição e Polimorfismo